TU Berlin
Institute of Mathematics
Professor Karow

TECHNISCHE UNIVERSITÄT BERLIN

Project in Numerical Analysis Summer 2026

Vortex Induced Vibration in Bladeless Turbine

Simulation and Analysis

Students: Hetvy Chanyiara, Petronella A. Kortleven, Nemanja Tesanoviç, Andrea Tomatis

1. Abstract

The global transition toward sustainable energy has prompted the development of novel wind harvesting systems. Among these, bladeless turbines represent a highly promising paradigm shift. Rather than relying on massive, rotating aerodynamic blades to capture wind energy, bladeless turbines consist of vertical, flexible masts that generate electricity by exploiting aeroelastic resonance. This report presents a comprehensive computational study of bladeless turbines utilizing a two-dimensional fluid-structure interaction framework. The numerical experiments developed in this study are explicitly designed to address three primary research questions. First, we investigate the aerodynamic flow conditions that maximize turbine efficiency, specifically analyzing the optimal Reynolds number for resonance and quantifying the detrimental impacts of realistic atmospheric shear flows compared to ideal uniform wind tunnel conditions. Second, we evaluate the structural domain to determine which material properties offer the highest theoretical energy yield by observing the interplay between structural mass, damping, and material stiffness. Third, we explore the aerodynamic synergy of densely packed wind farms by simulating the wake interference between multiple turbines positioned in tandem. By evaluating these phenomena, this study provides quantitative insights into the fundamental mechanics and optimal deployment strategies for vortex-induced vibration energy harvesters.

2. Introduction

The EU has committed to becoming climate-neutral by 2050 by reducing fossil fuels and harnessing energy from renewable sources. Wind power plays a key role in this goal, requiring the installed capacity to expand rapidly toward 2030 targets (European Commission, Joint Research Centre 2025). Europe currently possesses 304 GW of total wind capacity, with the EU-27 accounting for 246 GW of that infrastructure (WindEurope 2026). However, scaling traditional rotating wind infrastructure faces severe restrictions due to several reasons. Firstly, a looming material sustainability crisis—with old, non-recyclable blades projected to generate up to 10 million tonnes of waste by 2050 (European Commission, Joint Research Centre 2025). Additionally, there is increased urban grid congestion and severe avian collision risks across Europe (WindEurope 2026; Etard, Jung, and Visconti 2026). Bladeless turbines directly eliminate these bottlenecks by substituting massive rotating blades with a compact, solid-state vertical mast that limits avian collision, is silent, and can be manufactured using recyclable thermoplastic materials.

Bladeless turbines operate entirely on the principle of vortex-induced vibration. When a fluid flows past a slender bluff body, the boundary layer separates due to adverse pressure gradients, leading to the periodic shedding of vortices from alternating sides of the structure. This

phenomenon creates a predictable, highly structured wake pattern known in fluid dynamics as the Von Karman vortex street. The shedding of these vortices generates alternating low-pressure zones that exert a periodic lateral force on the structure perpendicular to the direction of the fluid flow. For maximum energy harvesting, the turbine structure must be carefully engineered such that its natural structural eigenfrequency matches the vortex shedding frequency dictated by the local wind speed. When these two frequencies align, the system enters a state of resonance known as lock-in. During the lock-in phase, the amplitude of the structural oscillation increases dramatically, allowing the internal linear alternators to convert the mechanical kinetic energy into usable electricity. If the wind speed drops too low, the shedding frequency cannot excite the structure. If the wind speed increases beyond the lock-in range, the shedding frequency outpaces the structure's physical capacity to respond, causing the vibration to dampen entirely.

In commercial deployments, an operational bladeless turbine consists of a rigid, three-dimensional tapered conical outer shell mounted atop an elastic internal carbon-fiber rod. This configuration acts as a rigid lever arm that tilts uniformly over a localized base pivot point (Vortex Bladeless 2026). Simulating this multi-body, 3D fluid-structure interface explicitly is computationally prohibitive for expansive parameter sweeps. Therefore, our framework simplifies the system into a two-dimensional side profile (X-Y plane) representing a homogeneous flexible column clamped at the base. To preserve the dynamics of the physical device within a 2D domain, a parabolic warping function is applied to the structural boundary mask. By scaling local horizontal grid displacements quadratically with height, the model ensures that structural mass distributions, elastic restoring forces, and ultimate kinetic power metrics accurately approximate the fundamental bending mode shape of a pivoting cantilever harvester.

Using this 2D framework, this project investigates what environmental and mechanical variables fundamentally affect this vortex shedding and resonance behavior. To do so, we establish three clear research objectives. First, we investigate the aerodynamic flow conditions that maximize turbine efficiency, specifically analyzing the optimal Reynolds number for resonance and quantifying the detrimental impacts of realistic atmospheric shear flows compared to ideal uniform wind tunnel conditions. Second, we evaluate the structural domain to determine which material properties offer the highest theoretical energy yield by observing the interplay between structural mass, damping, and material stiffness. Third, we explore the aerodynamic synergy of densely packed wind farms by simulating the wake interference between multiple turbines positioned in tandem.

3. Theoretical Foundations

(a) Fluid Dynamics Core

Traditional computational fluid dynamics solves the macroscopic Navier-Stokes equations for mass and momentum conservation. As fluid flows past a structural bluff body with velocity U, it sheds alternating vortices at a frequency dictated by the dimensionless Strouhal number St. The vortex shedding frequency f_v is defined as the Strouhal number multiplied by the flow velocity divided by the characteristic diameter D:

$$f_v = \frac{St \cdot U}{D} \tag{1}$$

These shedding vortices generate alternating low-pressure zones that exert an alternating transverse lift force on the pole.

(b) Structural Mechanics and Cantilever Dynamics

The physical mast of a bladeless turbine is modeled fundamentally as a continuous cantilever beam. The exact physical motion of this geometry under a continuous distributed load is

governed by the fourth-order partial differential Euler-Bernoulli beam equation:

$$EI\frac{\partial^4 y}{\partial x^4} + m_L\frac{\partial^2 y}{\partial t^2} + c\frac{\partial y}{\partial t} = F_{fluid}(x,t) \tag{2}$$

where EI is the flexural rigidity, m_L is the structural mass per unit length, and $y(x,t)$ is the horizontal deflection at height x. The structure possesses its own natural eigenfrequency f_n, defined mathematically as $1/(2\pi)$ multiplied by the square root of the stiffness divided by the mass:

$$f_n = \frac{1}{2\pi}\sqrt{\frac{k}{m}} \tag{3}$$

(c) Aeroelastic Coupling and Power Extraction

When the incoming wind velocity reaches a critical threshold where the shedding frequency matches this natural frequency ($f_v \approx f_n$), the system enters a state of resonance known as lock-in. Under these conditions, the aerodynamic lift forces phase-lock directly with the structural velocity, injecting maximum kinetic energy into the mast. The mechanical power $P(t)$ extracted by the internal linear alternator, represented in the differential equation by the damping coefficient c, is the product of the damping force and the structural velocity. Assuming the pole achieves a steady-state harmonic oscillation defined by maximum amplitude A and frequency f, the structural position over time is $\delta(t) = A\sin(2\pi ft)$. The instantaneous power is therefore proportional to the square of the derivative of this position. Integrating this instantaneous power over one complete oscillation cycle yields the average mechanical power:

$$P_{avg} = 2\pi^2 cA^2 f^2 \tag{4}$$

4. Computational Methodology and Implementation Framework

(a) The Lattice Boltzmann Method Fluid Solver

Our simulation employs the Lattice Boltzmann Method, which utilizes a mesoscopic approach based on kinetic gas theory. Instead of tracking continuous pressure and velocity fields across the computational domain, the Lattice Boltzmann Method tracks the probability distribution function of fictional fluid particles moving with discrete velocities. The foundation of this approach is the continuous Boltzmann equation coupled with the Bhatnagar-Gross-Krook collision approximation:

$$\frac{\partial f}{\partial t} + \vec{\xi}\cdot\nabla f = -\frac{1}{\tau}(f - f^{eq}) \tag{5}$$

where τ is the relaxation time parameter governing the kinematic fluid viscosity, and f^{eq} is the Maxwell-Boltzmann equilibrium distribution. To solve this computationally, two-dimensional space is discretized into a lattice grid, and particle velocities are strictly restricted to nine vectors using the standard D2Q9 lattice model. This spatial discretization restricts particle movement to nine specific velocity vectors extending from any given cell to its immediate neighbors and diagonals, including one rest particle that remains stationary. Each velocity direction i is associated with a specific discrete velocity vector $\vec{e}_i$ and a corresponding lattice weight w_i that ensures isotropic fluid behavior. In the Python implementation, these geometric lattice parameters are defined as multi-dimensional arrays to leverage highly vectorized operations across the entire grid simultaneously:

```
    def __init__(self, config: SimConfig, scene: Scene):
        # ... initialization code ...

        # D2Q9 Discrete Velocity Vectors (e_i)
        self.v = np.array([ [ 1,  1], [ 1,  0], [ 1, -1],
                            [ 0,  1], [ 0,  0], [ 0, -1],
```

```
                          [-1,  1], [-1,  0], [-1, -1] ])

        # D2Q9 Lattice Weights (w_i)
        self.t = np.array([ 1/36, 1/9, 1/36,
                            1/9,  4/9, 1/9,
                            1/36, 1/9, 1/36 ])
```

The computational simulation advances in discrete time steps through two fundamental operations known as collision and streaming. During the collision step, particle distributions within a single cell interact and relax toward a local thermal equilibrium state. We model this relaxation process using the Bhatnagar-Gross-Krook approximation, which introduces a single relaxation time parameter τ. In the code, this is represented by the relaxation frequency $\omega = 1/\tau$, which dictates the kinematic viscosity of the fluid. Before computing the collision, the macroscopic variables—specifically the fluid density ρ and the velocity vector $\vec{u}$—must be recovered. This is achieved by calculating the zeroth and first velocity moments of the probability distribution functions. Once the macroscopic fields are updated, the theoretical Maxwell-Boltzmann equilibrium distribution f_i^{eq} is calculated for every node. The polynomial expansion of this equilibrium state depends entirely on the local density and velocity. Furthermore, the fluid pressure p is calculated directly via an ideal gas equation of state, defined as $p = c_s^2\rho$, where c_s is the lattice speed of sound. This pressure differential across the structural boundary is integrated to continuously compute the driving aerodynamic forces.

```
    def _macroscopic(self):
        # Zeroth moment: Density is the sum of all particle distributions
        rho = np.sum(self.fin, axis=0)

        # First moment: Momentum is the sum of distributions * velocity vectors
        u = np.zeros((2, self.cfg.nx, self.cfg.ny))
        for i in range(9):
            u[0,:,:] += self.v[i,0] * self.fin[i,:,:]
            u[1,:,:] += self.v[i,1] * self.fin[i,:,:]

        # Velocity = Momentum / Density
        u /= rho
        return rho, u

    def _equilibrium(self, rho, u):
        usq = 3/2 * (u[0]**2 + u[1]**2)
        feq = np.zeros((9, self.cfg.nx, self.cfg.ny))
        for i in range(9):
            cu = 3 * (self.v[i,0]*u[0,:,:] + self.v[i,1]*u[1,:,:])
            # Maxwell-Boltzmann equilibrium polynomial expansion
            feq[i,:,:] = rho * self.t[i] * (1 + cu + 0.5*cu**2 - usq)
        return feq
```

Following the collision phase, the streaming step propagates the newly relaxed distribution functions to adjacent grid nodes strictly along their respective velocity vectors. A major computational advantage of the Lattice Boltzmann Method is that this advection process is strictly linear and requires no complex differential equation solvers. In our framework, streaming is executed instantaneously across the entire domain using the circular shift operator. Solid boundaries, such as the rigid wind tunnel walls and the dynamic flexible turbine structure, are handled via the bounce-back formulation. When a fluid particle collides with a solid grid node, its distribution is reflected precisely 180 degrees back to its origin. Because our velocity array is symmetrically constructed, the opposite vector for any direction i is elegantly accessed via the index $8 - i$. This effectively enforces the physical no-slip condition at the fluid-solid interface, even as the shape of the turbine warps dynamically over time.

```
    def step(self, iter_step):
        # ... boundary setup and macroscopic recovery ...

        # 1. Collision (Bhatnagar-Gross-Krook Approximation)
        # Particles relax toward equilibrium at a rate dictated by omega
        fout = self.fin - self.cfg.omega * (self.fin - feq)

        # 2. Environmental Boundaries (Wind Tunnel Walls)
        # Apply standard bounce-back to enforce no-slip conditions
        if self.cfg.top_bottom_walls:
            for i in range(9):
                # 8-i reverses the particle direction vector
                fout[i, :, 0] = self.fin[8-i, :, 0]         # Bottom Wall
                fout[i, :, -1] = self.fin[8-i, :, -1]       # Top Wall

        # 3. Dynamic Structural Boundaries (Turbine Array)
        # The scene evaluates the exact coordinates of all objects
        obstacle = self.scene.get_mask(self.X, self.Y, iter_step, rho, u)
        for i in range(9):
            fout[i, obstacle] = self.fin[8-i, obstacle]

        # 4. Streaming Phase
        # Propagate all distributions to their neighboring cells based on v[i]
        for i in range(9):
            self.fin[i,:,:] = np.roll(
                np.roll(fout[i,:,:], self.v[i,0], axis=0),
                self.v[i,1], axis=1
            )
        return rho, u, obstacle
```

(b) Lumped-Parameter Structural Solver

To optimize computational performance during the fluid-structure interaction sequence, a lumped-parameter approximation is applied. The continuous beam is mathematically reduced to a single point mass m supported by a theoretical spring with stiffness k and a damper with coefficient c. This transforms the complex partial differential equation into a solvable second-order ordinary differential equation:

$$m\ddot{\delta} + c\dot{\delta} + k\delta = F_{fluid}(t) \tag{6}$$

To ensure this ordinary differential equation accurately reflects a real macroscopic turbine of height L, width w, and wall thickness t_{wall}, the equivalent stiffness k is explicitly derived from Euler-Bernoulli theory for a tip-loaded cantilever. The area moment of inertia I is calculated as the structural width multiplied by the cubed thickness divided by twelve, yielding the final stiffness relation:

$$k = \frac{3EI}{L^3} \tag{7}$$

This derivation mathematically demonstrates that increasing the turbine height exponentially lowers its natural frequency, which is the primary mechanism by which bladeless turbines are tuned to extract energy from low-speed atmospheric winds.

The geometric module dictates how solid boundaries are defined and updated within the discrete lattice. A master scene container aggregates multiple arbitrary geometric objects, applying a logical OR operation across their respective boolean masks to generate a unified obstacle map. The most critical component of this module is the cantilever class, which models the bladeless turbine. At each time step, the turbine reads the local macroscopic fluid density directly from the solver. By sampling the lattice cells immediately adjacent to its upstream and downstream faces, it calculates the pressure differential. Because Lattice Boltzmann pressure is linearly proportional to density, this difference multiplied by a stabilization scaling factor yields the net aerodynamic fluid force.

```
# Inside FlexibleCantilever.get_mask()
up_x = max(0, int(self.cx - self.w))
down_x = min(rho.shape[0] - 1, int(self.cx + self.w + self.delta))
y_range = slice(int(self.cy_base), int(self.cy_base + self.h))

p_up = np.sum(rho[up_x, y_range]) / 3.0
p_down = np.sum(rho[down_x, y_range]) / 3.0
F_fluid = (p_up - p_down) * 0.01
```

This continuous fluid force is immediately passed into an explicit Euler numerical integrator. The integrator solves the second-order ordinary differential equation governing the lumped-parameter mass-spring-damper system. To maintain numerical stability and prevent mathematical explosions during resonance, strict clipping limits are enforced on both the velocity and the ultimate deflection.

```
acceleration = (F_fluid - self.k * self.delta - self.c * self.vel) / self.m
self.vel += acceleration
self.delta += self.vel

if self.delta > self.h/2:
    self.delta = self.h/2
    self.vel = 0.0
elif self.delta < -self.h/2:
    self.delta = -self.h/2
    self.vel = 0.0
```

Once the scalar deflection of the mast tip is computed, the object must update its physical footprint within the two-dimensional grid. To accurately simulate the bending of a continuous cantilever beam subjected to a distributed load, the geometric boolean mask is warped. The horizontal shift of any given vertical point follows a parabolic curve, normalized against the total height of the structure.

```
Y_norm = np.clip((Y - self.cy_base) / self.h, 0, 1)
dX = self.delta * (Y_norm ** 2)

y_bounds = (Y >= self.cy_base) & (Y <= self.cy_base + self.h)
x_bounds = np.abs(X - (self.cx + dX)) <= self.w / 2

self.last_mask = y_bounds & x_bounds
```

The solver module encapsulates the Lattice Boltzmann Method algorithms. While the mathematical theory of the D2Q9 lattice and the Bhatnagar-Gross-Krook collision operator were detailed in the preceding section, the software implementation relies heavily on multi-dimensional array vectorization. Instead of iterating through individual grid cells using nested loops, the solver leverages NumPy element-wise operations to compute the macroscopic variables and the collision relaxations simultaneously across the entire domain. Within the primary solver step, the temporal sequence is strictly ordered: macroscopic variable recovery is followed by inflow condition enforcement, equilibrium calculation, collision relaxation, boundary condition application, and finally, spatial streaming. The dynamic obstacle mask generated by the geometry module is integrated directly after the collision step, applying the bounce-back reflection condition specifically to the cells currently occupied by the moving turbine.

```
# Collision step
fout = self.fin - self.cfg.omega * (self.fin - feq)

# Fetch the dynamic geometry mask
obstacle = self.scene.get_mask(self.X, self.Y, iter_step, rho, u)
```

```
# Apply bounce-back to the moving obstacle
for i in range(9):
    fout[i, obstacle] = self.fin[8-i, obstacle]
```

(c) Simulation Configuration and Execution (`main.py`)

The entry point of the framework is responsible for handling the global simulation state and orchestrating the temporal loop. To guarantee reproducibility across experiments, all physical and environmental parameters are encapsulated within a strongly typed data structure. This configuration object acts as the single source of truth for the lattice dimensions, Reynolds number, characteristic length, and output frequencies. It also dynamically computes derived lattice properties, such as the kinematic viscosity and the corresponding collision relaxation frequency.

```
@dataclass
class SimConfig:
    nx: int = 600
    ny: int = 200
    Re: float = 250.0
    uLB: float = 0.04
    max_iter: int = 5000
    L_char: float = 20.0
    top_bottom_walls: bool = False
    flow_profile: str = 'uniform'

    @property
    def nulb(self):
        return self.uLB * self.L_char / self.Re

    @property
    def omega(self):
        return 1.0 / (3.0 * self.nulb + 0.5)
```

The primary execution loop iteratively advances the fluid solver and processes the resulting data. At specified intervals, the loop extracts the maximum fluid velocity and the structural deflection of every object present in the scene, appending this data to a comma-separated values log. Concurrently, it computes the fluid vorticity curl and maps it to a color matrix, rendering the physical state into a compressed video file via the OpenCV library.

(d) Automated Testing and Batch Processing (`test.py`)

To conduct rigorous scientific parameter sweeps without manual intervention, the testing module programmatically generates JavaScript Object Notation configuration files. This automated suite orchestrates the execution of four distinct experimental phases, each designed to isolate and evaluate specific physical variables governing bladeless turbine performance.

(d).1 Phase 1: Aeroelastic Lock-in Sweep

The primary objective of the first testing phase is to empirically identify the exact lock-in resonance curve for a baseline turbine geometry. To achieve this, the test suite generates a batch of simulations where the structural properties, specifically the mass and stiffness, are held strictly constant. The simulation utilizes stable mathematical parameters, setting the structural stiffness to 0.008, the mass to 3.0, and the damping coefficient to 0.002. The overarching loop systematically increments the fluid Reynolds number across five distinct steps: 150, 200, 250, 300, and 350.

```
# Systematic parameter sweep to identify the resonance peak
simulations = []
for re in [150, 200, 250, 300, 350]:
```

```
    name = f"P1_Re_{re}"
    simulations.append({
        "name": name,
        "re": float(re),
        "flow": "uniform",
        % Structural parameters remain constant across all iterations
        "objects": [{"shape": "flexible_pole", "stiffness": 0.008,
                     "mass": 3.0, "damping": 0.002}]
    })
```

By generating discrete outputs for each velocity step spanning this range, the phase ensures the fluid's Strouhal shedding frequency crosses the structure's natural eigenfrequency. The inclusion of the damping parameter prevents infinite amplitude blow-up at the exact lock-in threshold, allowing the analyzer to capture the exponential rise and stable plateau of the harmonic oscillation amplitude.

(d).2 Phase 2: Material Optimization Matrix

Once the optimal lock-in Reynolds number is established (fixed at 250.0 for this phase), the second module evaluates the structural domain. A critical engineering function of this module is non-dimensionalization. Raw macroscopic material properties cannot be directly ingested by the explicit fluid-structure interaction solver without causing catastrophic numerical instability. Therefore, the testing module defines a matrix of four real-world materials: polyvinyl chloride plastic, fiberglass, aluminum, and carbon fiber. The routine calculates the true area moment of inertia and physical stiffness for an eight-meter-tall mast based on the specific Young's Modulus and density of each material. For instance, it accounts for the massive rigidity difference between carbon fiber (150 gigapascals) and PVC plastic (3 gigapascals). It then multiplies these real-world values by constant scaling factors, specifically 10^{-6} for stiffness and 10^{-5} for mass.

```
# Real-world material properties mapped to stable LBM units
materials = {
    "PVC_Plastic":  {"E": 3.0e9,   "rho": 1380},
    "Fiberglass":   {"E": 15.0e9,  "rho": 1900},
    "Aluminum":     {"E": 69.0e9,  "rho": 2700},
    "Carbon_Fiber": {"E": 150.0e9, "rho": 1600}
}

scale_k = 1e-6
scale_m = 1e-5

for mat_name, props in materials.items():
    real_k = (3 * props["E"] * I) / (height_m ** 3)
    lbm_stiffness = real_k * scale_k
    lbm_mass = (props["rho"] * volume) * scale_m
```

This scaling methodology strictly preserves the relative density and elasticity ratios between the materials, allowing the solver to accurately simulate how a heavily rigid material behaves under identical aerodynamic loads compared to a highly elastic material without shattering the discrete time integration.

(d).3 Phase 3: Atmospheric Boundary Layer Impact

The third experimental phase transitions the testing environment from an idealized wind tunnel to a realistic atmospheric scenario. Real-world wind does not strike vertical structures uniformly; it creates a shear boundary layer where velocity increases continuously with altitude. The automated script iterates over a binary list to toggle the computational domain's inflow condition between a uniform profile and a shear gradient.

```
# Toggling the inflow boundary conditions for environmental testing
for flow_type in ['uniform', 'shear']:
    name = f"P3_Flow_{flow_type.capitalize()}"
    simulations.append({
        "name": name,
        "flow": flow_type,
        "objects": [{"shape": "flexible_pole", "cx": 200, "cy": 1,
                     "width": 10, "height": 100,
                     "stiffness": 0.005, "mass": 2.0}]
    })
```

Both simulations evaluate an identical generic flexible mast over 8000 computational steps. This comparative test is critical for quantifying the destructive interference caused by shear flow, demonstrating how variations in local shedding frequencies along the vertical axis of the mast degrade the overall energy harvesting efficiency.

(d).4 Phase 4: Multi-Turbine Wake Interference

The final testing phase scales the simulation from a single isolated mast to a high-density wind farm layout. To evaluate aerodynamic synergy and wake-induced flutter, the configuration generator places four independent flexible cantilevers into an in-line tandem array. To mimic actual operational conditions, this phase exclusively utilizes the shear flow boundary condition.

```
# Generating a highly dense multi-mast array layout
"objects": [
    {"shape": "flexible_pole", "cx": 200, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 450, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 700, "stiffness": 0.006, "mass": 2.5},
    {"shape": "flexible_pole", "cx": 950, "stiffness": 0.006, "mass": 2.5}
]
```

The turbines are uniformly spaced 250 spatial units apart along the longitudinal axis, occupying a significantly extended computational domain of 1100 by 300 cells. Because the scene geometry module dynamically aggregates all object masks, the underlying Lattice Boltzmann solver naturally resolves the complex fluid mechanics of the upstream vortices impacting the downstream structures. This layout enables the data analyzer to directly compare the oscillation phases and power yields of the wake-embedded turbines against the primary upstream mast over an extended duration of 12000 steps.

(e) Data Extraction and Performance Analysis (`analysis.py`)

The final module functions as the post-processing pipeline, converting raw discrete time-series data into continuous performance metrics and publication-ready visualizations. Utilizing the Pandas library, the analyzer ingests the structural logs generated by the batch tests. To ensure accurate steady-state harmonic analysis, the script isolates a specific analytical window, explicitly ignoring the transient fluid spin-up and initial structural startup phases. To evaluate the comparative efficiency of different turbine configurations, the analyzer extracts both the maximum continuous amplitude and the predominant oscillation frequency. The frequency is computed algorithmically by identifying the indices where the derivative of the positional sign changes, which precisely isolates the structural zero-crossings across the neutral axis. Half the number of these zero-crossings divided by the elapsed simulation steps yields the normalized frequency. Because the constant multiplier containing the geometric and damping factors is applied uniformly across all simulated materials during comparative testing, it is factored out to form the dimensionless power proxy utilized exclusively in this study's analysis phase. The power proxy is defined strictly as the squared amplitude multiplied by the squared frequency. This formulation mathematically proves that an excessively stiff material may ultimately generate

less total electrical power than a moderately flexible material, as extreme structural rigidity severely limits the oscillation amplitude despite allowing for higher vibrational frequencies.

```
# Isolate steady-state oscillation
plot_data = df[(df['Step'] > 2000) & (df['Step'] < 8000)]

# Calculate Amplitude (Mean of absolute peaks)
amplitude = plot_data['Deflection_dX'].abs().mean()

# Calculate Frequency via zero-crossings
zero_crossings = np.where(np.diff(np.sign(plot_data['Deflection_dX'])))[0]
num_cycles = len(zero_crossings) / 2.0
time_steps = plot_data['Step'].max() - plot_data['Step'].min()
frequency = num_cycles / time_steps

% Compute theoretical mechanical energy
power_proxy = (amplitude ** 2) * (frequency ** 2)
```

These extracted values are then fed into the normalized mechanical power proxy equation discussed in the introduction. The module dynamically generates comparative line plots of the positional waveforms, evaluates wake-induced efficiency ratios for multi-mast arrays, and exports a fully ranked numerical leaderboard of material performance to a secondary formatted dataset.

5. Results

TODO.

(a) Conclusions

TODO.

References

European Commission, Joint Research Centre (2025). *Circular Economy Strategies for the EU's Renewable Electricity Supply*. CEPRES Exploratory Research Project Report. Luxembourg: Publications Office of the European Union. URL: https://publications.jrc.ec.europa.eu/repository/handle/JRC138570.

WindEurope (Feb. 2026). *Wind energy in Europe: 2025 Statistics and the outlook for 2026-2030*. Brussels, Belgium: WindEurope. URL: https://windeurope.org/data/product/wind-energy-in-europe-2025-statistics-and-the-outlook-for-2026-2030/.

Etard, A., M. Jung, and P. Visconti (2026). "Risk to European birds from collisions with wind-energy facilities". In: *Biological Conservation* 319, e111875. DOI: 10.1016/j.biocon.2026.111875. URL: https://doi.org/10.1016/j.biocon.2026.111875.

Vortex Bladeless (2026). *Vortex Bladeless Wind Energy: Vortex-Induced Vibration Aerogenerators*. Official Company Website. URL: https://vortexbladeless.com/.